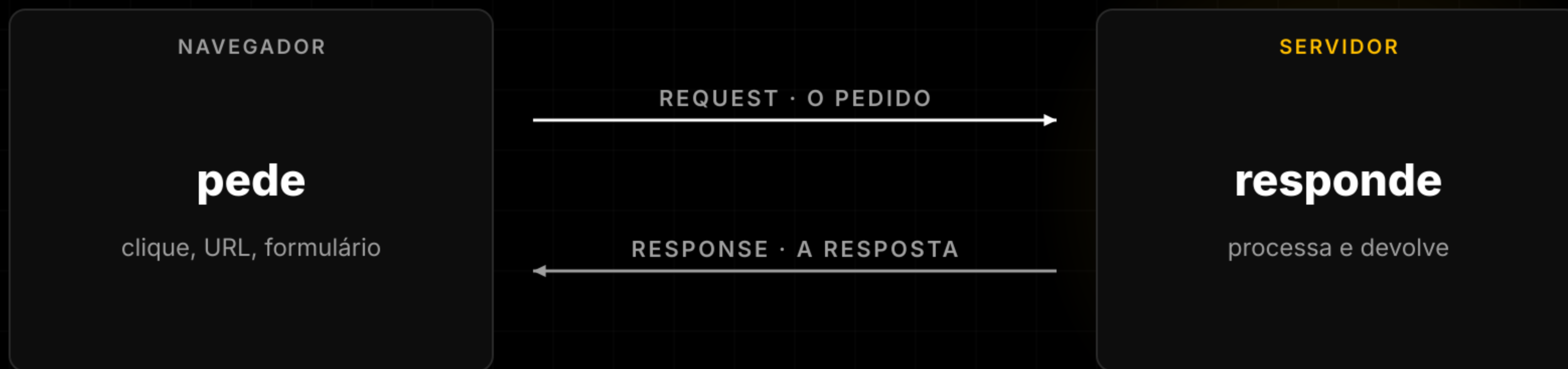


Entre o clique e a tela, uma conversa



O navegador pede. O servidor responde. Sempre nessa ordem.

GET pede. POST envia.

GET · PEDIR

"me dá isso"

abrir a página, ver a lista,
carregar o produto

não muda nada no servidor

POST · ENVIAR

**"aqui estão dados,
faz algo com isso"**

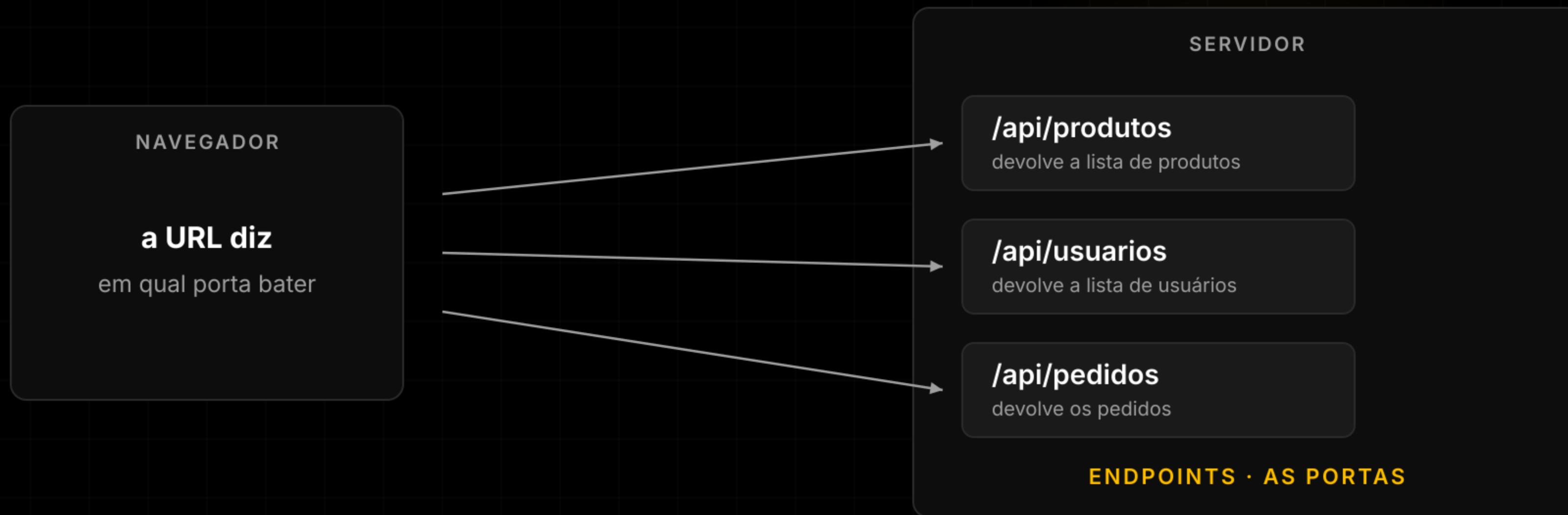
criar a conta, enviar o formulário,
adicionar ao carrinho

manda dados que mudam algo

Existem outros (PUT, PATCH, DELETE), mas esses dois cobrem 90% da web.

Se a IA diz "fazer um GET", ela está dizendo o tipo do pedido.

Endpoint: cada URL é uma porta



Você não pede sanduíche no caixa eletrônico. Cada porta atende um pedido.

JSON: a resposta em texto organizado

RESPONSE · O QUE VOLTA

```
{  
  "nome": "Produto A",  
  "preco": 29.90,  
  "estoque": true  
}
```

TEXTO QUE O FRONT LÊ

chaves { } agrupam os campos

abrem e fecham um bloco de dados

aspas marcam textos

números e verdadeiro/falso vão sem aspas

cada linha: um nome e um valor

o front lê isso e monta a tela

Fácil para o computador ler. Fácil para você conferir.

O contrato: o que peço, o que recebo

CONTRATO CERTO

front pede: um produto

servidor devolve:

```
{ "nome": ..., "preco": ... }
```

front lê nome e preço

✓ funciona

CONTRATO QUEBRADO

front espera: título

servidor devolve:

```
{ "nome": ..., "preco": ... }
```

front procura título, não acha

✗ tela em branco

Contrato: o combinado entre front e servidor. O que eu peço, e o que recebo.

Se um lado muda o combinado, o outro quebra.

Quando a IA inventa o contrato

O QUE A IA ESCREVEU

o front espera receber:

"título": ...

"descrição": ...

campos que ninguém confirmou



O QUE O SERVIDOR DEVOLVE

o contrato real:

"nome": "Produto A"

"preco": 29.90

nada de título ou descrição

A IA CHUTOU OS CAMPOS

O código roda sem acusar erro. E a tela fica vazia.

Pergunte: "você viu o contrato real, ou está chutando os campos?"

Cinco status codes cobrem quase tudo

VERDE: DEU CERTO · ÂMBAR: PROBLEMA NO PEDIDO · VERMELHO: QUEBROU LÁ DENTRO

200

OK

deu certo, aqui está a resposta

404

NOT FOUND

não achei o que você pediu

401

UNAUTHORIZED

você não está logado: não sei quem você é

403

FORBIDDEN

sei quem você é, mas você não pode acessar isso

500

SERVER ERROR

quebrou lá dentro: não foi culpa sua

O status code é o servidor sendo honesto sobre o que aconteceu.

401 e 403: portas fechadas diferentes

401

UNAUTHORIZED

“não te conheço”

falta login: quem é você?

403

FORBIDDEN

“te conheço, e não pode”

logado, sem permissão para isso

A PORTA FECHOU PELO MOTIVO CERTO?

401 pede login. 403 pede permissão.

Crash honesto, crash silencioso

RESPOSTA HONESTA



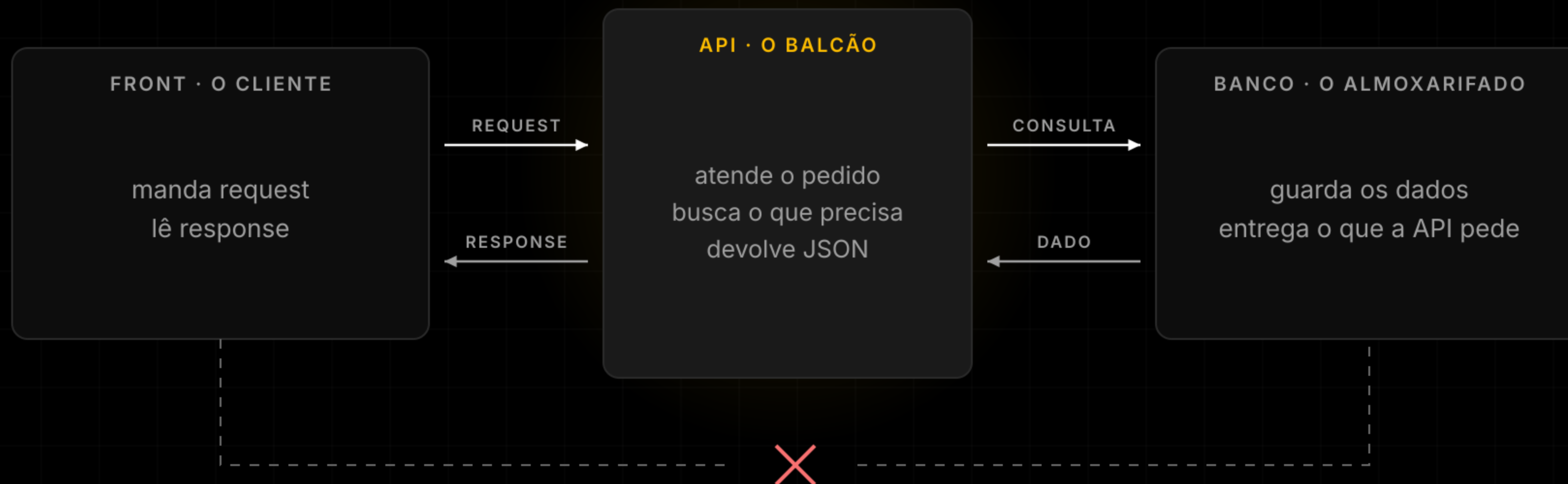
CRASH SILENCIOSO



Quando a IA falar em "tratar erro", pergunte: tratar qual? 404, 401, 500?

O pior erro não é o 500. É a resposta que não diz nada.

A fronteira: o front não vê o banco



O FRONT NÃO FALA DIRETO COM O BANCO

Você pede no balcão. Não entra no almoxarifado.

Camadas: cada uma com um trabalho



Separadas, você troca uma sem quebrar a outra. Misturadas, tudo quebra junto.

PERGUNTE À IA: ISSO MORA EM QUAL CAMADA?

Cada camada tem um trabalho. A fronteira mantém o sistema são.

As três armadilhas da IA com API

ARMADILHA 1

inventar contrato

espera campos que o servidor
nunca confirmou

PERGUNTE

*"Você viu o contrato
ou chutou?"*

ARMADILHA 2

ignorar status code

trata 404, 401 e 500
como se fossem o mesmo erro

PERGUNTE

*"Qual status code
eu devo tratar?"*

ARMADILHA 3

misturar camadas

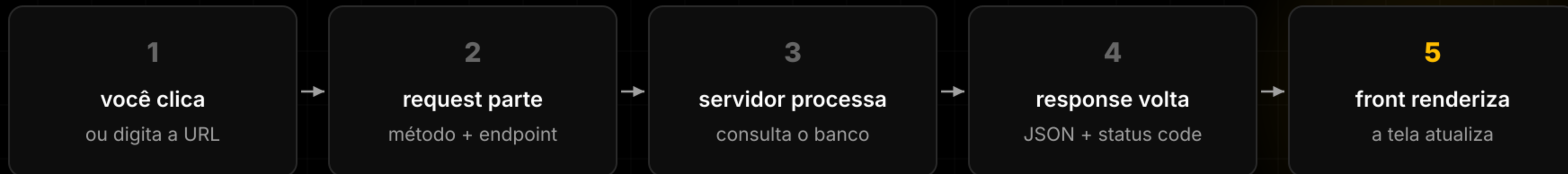
põe lógica de banco no front,
ou de front no servidor

PERGUNTE

*"Isso mora em
qual camada?"*

Três perguntas, um crivo. Faça-as toda vez que a IA mexer com API.

A conversa inteira, do clique à tela



Esse ciclo é a conversa inteira. E ela acontece a cada clique.

O front pede. A API atende. O banco guarda. Cada um no seu lugar.